All questions in the exam have one (1) and only one correct alternative.

Answering with more than one alternative for a question will result in zero (0) points for the question.

General instruction for understanding the code

Unless the question states differently, understand the code in this exam as:

1. Javascript code
2. each question has its own javascript code file, that is run using node, e.g. think of the code for a question as: - its all in own file (`question8.js`), and - it is executed using the command `node question8.js`
3. the latest javascript standard.

Questions

Section 1: Javascript basics

1. Which feature does JavaScript support? `(1p)`

a) First-class structs
b) functions as values
c) modules as values

2. What is the difference between == and === in JavaScript? `(1p)`

a) Both are strict equality checks
b) `==` is a loose equality, `===` is strict

c) Both are loose equality checks

3. What concept in JavaScript is foundational for asynchronous operations? (1p)

a) Generators
b) Coroutines
c) Callback functions

4. What kind of inheritance does JavaScript support? (1p)

a) Classical inheritance
b) Prototypical inheritance
c) Structural inheritance

5. Which operator in JavaScript can be used to expand arrays and other expressions? (1p)

a) `*..*`
b) `+++`
c) `...`

6. Which function can you use to determine if a variable is an array in JavaScript? (1p)

a) `Object.isType()`
b) `Array.isArray()`
c) `Array.isObject()`

7. Which of the following is not a state of a Promise in JavaScript? (1p)

a) Fulfilled
b) Executing
c) Rejected

8. What kind of functions does not have a name in JavaScript? (1p)

a) Named functions
b) Recursive functions
c) Anonymous functions

9. Which kind of functions do not have their own `this` value in JavaScript? (1p)

a) Normal functions
b) Constructor functions
c) Arrow functions

10. What does an IIFE stand for in JavaScript? (1p)

a) Instantly Invoked Function Example
b) Internally Initiated Function Execution
c) Immediately Invoked Function Expression

Section 2: Types and Type conversion

11. Given a variable that has the value:

```
null
```

What is the resulting value when converted to a *number*? (2p)

1. undefined
2. null
3. true
4. false
5. ""
6. "1.2"
7. 1.2
8. "one"
9. 0
10. 1
11. Infinity
12. -Infinity
13. NaN

12. Given a variable that has the value:

```
true
```

What is the resulting value when converted to a *number*? (2p)

1. undefined
2. null
3. true
4. false
5. ""
6. "1.2"
7. 1.2
8. "one"
9. 0
10. 1
11. Infinity
12. -Infinity
13. NaN

13. Given a variable that has the value:

```
    null
```

What is the resulting value when converted to a *boolean*? (2p)

1. undefined
2. null
3. true
4. false
5. ""
6. "1.2"
7. 1.2
8. "one"
9. 0
10. 1
11. Infinity
12. -Infinity
13. NaN

14. Given a variable that has the value:

```
        -Infinity
```

What is the resulting value when converted to a *boolean*? (2p)

1. undefined
2. null
3. true
4. false
5. ""
6. "1.2"
7. 1.2
8. "one"
9. 0
10. 1
11. Infinity
12. -Infinity
13. NaN

Section 3: Functions

15. Given the following JavaScript code:

```
function welcome(msg) {
    return "Welcome, " + msg + "!";
}
```

Which of the following will print "Welcome, Student!"? (2p)

a) `welcome()`
b) `welcome(Student)`
c) `welcome("Student")`
d) `welcome.Welcome("Student")`

16. Which of the following is NOT a correct way to call a function named `display` in JavaScript? (2p)

a) `display.call();`
b) `display();`
c) `display.invoke();`
d) `new display();`

17. Given the JavaScript function:

```
function personDetails(firstName, lastName = "Smith") {
    return `${firstName} ${lastName}`;
}
```

What will the output be for `personDetails("John")`? (2p)

a) `John`
b) `John undefined`
c) `John Smith`
d) `Smith John`

18. Given the following JavaScript function:

```
function multiply(...vals) {
    return vals.reduce((acc, curr) => acc * curr, 1)
}
```

Which two of the following will return the value **24**? (2p)

a) `multiply([2, 3, 4])`
b) `multiply(...[2, 3, 4])`
c) `multiply(2, 3, 4)`
d) `multiply({2, 3, 4})`

19. Consider the code:

```
let a = function(b) {
    return b + b
}

let c = a;
```

What will `c(5)` return? `(2p)`

a) `55`
b) `10`
c) `25`
d) `5`

Section 4: Exceptions, throw, try/catch/finally

20. After executing the code below, what will be the value of `total`? `(2p)`

```
let total = 5;

try {
    total = total * 2;
    throw new Error();
} catch (err) {
    total = total + 3;
} finally {
    total = total - 2;
}
```

a) 8
b) 10
c) 11
d) 6

21. What type of error does the following code throw? `(2p)`

```
let data = null;
console.log(data.length);
```

a) ReferenceError
b) TypeError
c) SyntaxError
d) RangeError

Section 5: String

22. What will be the output of the following JavaScript code? `(1p)`

```
let str = "OpenSource"
console.log(str.substring(4, 7))
```

a) “Source”

b) “Sou”
c) “enSo”
d) “enS”

23. What does the following code return? (1p)

```
"JavaScript".startsWith("Java")
```

a) true
b) false
c) “Java”
d) “Script”

24. Consider the following code:

```
let str1 = "Hello"
let str2 = "World"
let result = str1.concat(" ", str2)
console.log(result)
```

What will be the output? (1p)

a) “HelloWorld”
b) “Hello World”
c) “Hello”
d) “World”

25. What will be the output of the following JavaScript code? (1p)

```
let phrase = "FrontEnd Development"
console.log(phrase.indexOf("End"))
```

a) 4
b) 5
c) 6
d) -1

Section 6: Loops (for, for/of, for/in, while, do/while)

26. How many times will the message “Hello World!” be printed? (2p)

```
let i = 0
do {
    console.log("Hello World!")
    i++
} while(i < 3)
```

a) 2 times
b) 3 times
c) 4 times
d) 0 times

27. Which of the following code alternatives will print the message "Hello" exactly once? (2p)

A)

```
let i = 5
while (i < 5) {
    console.log("Hello")
}
```

B)

```
let j = 5
do {
    console.log("Hello")
} while (j < 5)
```

C)

```
let k = 6
while (k <= 5) {
    console.log("Hello")
}
```

D)

```
let l = 4
do {
    console.log("Hello")
} while (l <= 4 && l++)
```

28. What does the following code output? (2p)

```
let fruits = ['apple', 'banana', 'cherry']
for (let fruit of fruits) {
    console.log(fruit)
}
```

a) apple, banana, cherry (on different lines)
b) 0, 1, 2 (on different lines)
c) applebananacherry (on a single line)
d) an error

29. Consider the following code:

```
let data = [10, 20, 30, 40]
let result = 0
for (let item in data) {
    result += data[item]
}
console.log(result)
```

What will be the output? (2p)

a) 100
b) 10
c) "0123"
d) 1000

30. What will be the output of the following code? (2p)

```
let output = ""

loop1:
for (let i = 1; i <= 2; i++) {
    loop2:
    for (let j = 1; j <= 2; j++) {
        if (j === 2) {
            continue loop1
        }
        output += `${i}${j} `
    }
}
console.log(output.trim())
```

a) "11 12 21 22"
b) "11 21"
c) "12 22"
d) "11 12"

Section 7: Arrays and its functions

31. What does the following code output? (2p)

```
const fruits = ['apple', 'banana', 'cherry']
console.log(fruits.push('date'))
```

a) 3
b) 4
c) 'date'

d) undefined

32. What is the output of the following code snippet? (2p)

```
const colors = ['red', 'blue', 'green']
console.log(colors.length)
```

a) 2
b) 3
c) 4
d) 'blue'

33. After executing the following code, what the output? (2p)

```
const nums = [1, 2, 3]
nums.unshift(0)
console.log(nums)
```

a) [0, 1, 2]
b) [1, 2, 3, 0]
c) [1, 2, 3]
d) [0, 1, 2, 3]

34. What will be the output of the following code snippet? (2p)

```
const array = [5, 10, 15, 20]
console.log(array.shift())
```

a) 20
b) [5, 10, 15, 20]
c) 5
d) [10, 15, 20]

Section 8: Objects

35. Dot notation vs bracket notation

```
const car = {
    model: "Tesla"
}
console.log(car["model"])
```

What is the output of the code above? (2p)

a) "Tesla"
b) model
c) undefined

d) car

36. Checking for properties (in operator)

```
const animal = {
    type: "Dog"
}
console.log("color" in animal)
```

What is the output of the code above? (2p)

a) true
b) false
c) “Dog”
d) undefined

37. Object.keys()

```
const student = {
    name: "Alex",
    age: 20
}
console.log(Object.keys(student).length)
```

What is the output of the code above? (2p)

a) 2
b) “Alex”
c) 20
d) 1

38. Context (this keyword)

```
const person = {
    name: "Mike",
    sayHello: function() {
    return "Hello, " + this.name
    }
}
const greet = person.sayHello
console.log(greet())
```

What is the output of the code above? (2p)

a) “Hello, Mike”
b) “Hello, undefined”
c) “Hello, this.name”

d) “Hello, person”

Section 9: Prototypes and classes

39. Given the following JavaScript code:

```
function Animal(name) {
    this.name = name
}

Animal.prototype.speak = function() {
    return `${this.name} makes a sound`
}

function Dog(name) {
    Animal.call(this, name)
}

Dog.prototype = Object.create(Animal.prototype)
Dog.prototype.constructor = Dog

Dog.prototype.speak = function() {
    return `${this.name} barks`
}

let max = new Dog("Max")

console.log(max.speak())
```

What will be the output of the code snippet? (2p)

a) “Animal makes a sound”
b) “Max barks”
c) “Max makes a sound”
d) “Dog barks”

40. Given the following JavaScript code:

```
function Person(name) {
    this.name = name
}

Person.prototype.sayHi = function() {
    return `Hello, ${this.name}!`
}
```

```
let person1 = new Person("Alice")
let person2 = Object.create(person1)

console.log(person2.sayHi())
```

What will be the output of the code snippet? (2p)

a) "Hello, Alice!"
b) "Hello, !"
c) "Hello, undefined!"
d) Error

41. Given the following ES6 class definition:

```
class Cat {
    constructor(name) {
        this.name = name
    }

    sound() {
        return `${this.name} meows`
    }
}

let whiskers = new Cat("Whiskers")
console.log(whiskers.sound())
```

What will be the output of the code snippet? (2p)

a) "Cat meows"
b) "Whiskers barks"
c) "Whiskers meows"
d) "Sound meows"

Section 10: Iterators & generators

42. What will be the output of the following code? (2p)

```
let numbers = [8, 9, 10]
let iterator = numbers.entries()

let secondOutput = iterator.next().value
secondOutput = iterator.next().value
console.log(secondOutput)
```

a) [1, 9]
b) [0, 8]

c) 9
d) [9, 1]

43. What will be the output of the following code? (2p)

```
let fruits = ['apple', 'banana', 'cherry']
let iterator = fruits.keys()

let thirdKey = iterator.next().value
thirdKey = iterator.next().value
thirdKey = iterator.next().value
console.log(thirdKey)
```

a) 'apple'
b) 0
c) 'banana'
d) 2

44. What will be the output of the following code? (2p)

```
function* numberGenerator() {
    yield 5 * 5
    yield 6 * 6
    yield 7 * 7
}

let gen = numberGenerator()
let secondNumber = gen.next().value
secondNumber = gen.next().value
console.log(secondNumber)
```

a) 25
b) 36
c) 49
d) 5

45. What will be the output of the following code? (2p)

```
function* alternatingGenerator() {
    let data = [10, 20];
    let idx = 0;

    while (true) {
        yield data[idx];
        idx = 1 - idx;  // This will alternate idx between 0 and 1
```

```
    }
}

let sequence = alternatingGenerator()
let fourthValue = sequence.next().value
fourthValue = sequence.next().value
fourthValue = sequence.next().value
fourthValue = sequence.next().value
console.log(fourthValue)
```

a) 10
b) 20
c) 30
d) 40

Section 11: Modules

46. ES6 Module Import/Export

Consider the following files:

calcFunctions.js:

```
export const multiply = (a, b) => a * b
export const divide = (a, b) => a / b
```

mainApp.js:

```
import { multiply, divide } from './calcFunctions.js'

let x = multiply(2, 5)
let y = divide(8, 2)
console.log(x,y)
```

Based on the provided code snippets, what will be the output when `mainApp.js` is executed? `(2p)`

a) `10 4`
b) `10 6`
c) `7 4`
d) `6 4`

47. Default Exports

Consider the following files:

messageModule.js:

```
const message = "Good Morning!"
export default message
```

appMain.js:

```
import morningMessage from './messageModule.js'

console.log(morningMessage)
```

What will be the output when `appMain.js` is executed? `(2p)`

a) `Good Morning!`
b) `messageModule`
c) `morningMessage`
d) `undefined`

Section 12: Std library

48. `map()` Function

Consider:

```
const words = ['apple', 'banana', 'cherry']
const lengths = words.map(word => word.length)
console.log(lengths)
```

What is the output? `(1p)`

a) [‘apple’, ‘banana’, ‘cherry’]
b) [5, 6, 6]
c) 17
d) 3

49. `reduce()` Function

Given:

```
const nums = [2, 3, 4]
const product = nums.reduce((total, n) => total * n)
console.log(product)
```

What is the printed result? `(1p)`

a) 9
b) 6
c) 24
d) [2, 3, 4]

50. `filter()` Function

Given:

```
const characters = ['A', 'b', 'C', 'd', 'E']
const upperChars = characters.filter(char => char === char.toUpperCase())
console.log(upperChars)
```

What is the output? `(1p)`

a) [‘A’, ‘C’, ‘E’]
b) [‘A’, ‘b’, ‘C’, ‘d’, ‘E’]
c) [‘b’, ‘d’]
d) []

51. `forEach()` Function

Given:

```
const numbersArray = [1, 2, 3]
let multipliedByThree = []
numbersArray.forEach(num => multipliedByThree.push(num * 3))
console.log(multipliedByThree)
```

What is the printed value? `(1p)`

a) [1, 2, 3]
b) [3, 6, 9]
c) 6
d) [2, 4, 6]